Fiche de révision — tout le code de Miaoumi

Idir, Ama et Maria doivent savoir expliquer les trois modules · Bachelor 3e année · état du code au 07/11/2026 (branche develop)

1. Le projet en une phrase

Miaoumi met en relation des **propriétaires** d'animaux et des **gardiens** : un propriétaire publie une annonce de garde, des gardiens postulent, le propriétaire accepte ou refuse, il clôture la garde, puis chacun laisse un avis. Un même compte peut être propriétaire et gardien. PHP sans framework, MySQL, architecture MVC.

2. La structure des dossiers

```
miaoumi/
├── config/        database.php (connexion PDO unique), config.example.php, config.php (local, ignoré par Git)
├── app/
│   ├── core/      bootstrap.php (session, chargements, e(), CSRF), flash.php
│   ├── models/    Utilisateur, Annonce, Candidature, Avis      → SQL + règles métier
│   ├── controllers/  Auth, Annonce, Candidature, Avis          → lit la requête, décide
│   └── views/     pages HTML + header, footer, navigation      → affichage seulement
├── public/        index.php (seule porte d'entrée) + assets/ (CSS, logo, favicon)
├── sql/           miaoumi.sql (schéma), donnees-demo.sql, migrations/
└── tests/         annonces.php (tables MySQL temporaires)
```

Couche	Rôle	Règle à retenir
Modèle	Parle à la base, applique les règles métier	Valide **avant** d'écrire en base
Contrôleur	Lit $_GET / $_POST / session, appelle le modèle, affiche une vue ou redirige	Ne contient pas de HTML
Vue	Fabrique le HTML	Aucune requête SQL ; tout texte passe par e()

À dire à l'oral : « Le modèle gère les données, le contrôleur fait le lien, la vue affiche. Chacun a un seul rôle, donc le code est plus clair et on peut travailler à trois sans se gêner. »

3. Le chemin d'une requête

- Toutes les pages passent par `public/index.php` : c'est le **front controller**.
- Il charge `app/core/bootstrap.php` : `session_start` (cookie httponly, samesite=Lax, strict_mode), chargement des classes, fonction `e()`, création du jeton CSRF et **vérification du jeton à chaque POST** (sinon 403).
- Le **routeur** cherche `$_GET['page']` dans le tableau `$routes` et appelle la méthode du contrôleur.
- Page inconnue → 404 avec la page « introuvable » ; base injoignable (PDOException) → 503 avec la page « indisponible ».

```
$routes = [
    'annonces' => [AnnonceController::class, 'index'],
    'postuler' => [CandidatureController::class, 'postuler'],  ...
];
$page = $_GET['page'] ?? 'annonces';              // page par défaut
[$classe, $methode] = $routes[$page];
(new $classe())->$methode();                      // ex : (new CandidatureController())->postuler()
```

?page=	Méthode appelée	?page=	Méthode appelée
annonces, mes-annonces	AnnonceController::index	postuler	CandidatureController::postuler
creer-annonce	AnnonceController::creer	decision-candidature	CandidatureController::decision
cloturer-annonce	AnnonceController::cloturer	avis	AvisController::index

?page=	Méthode appelée	?page=	Méthode appelée
candidatures, mes-candidatures, candidatures-recues	CandidatureController::index	inscription, connexion, deconnexion	AuthController::…

À dire à l'oral : « Toutes les requêtes arrivent sur index.php ; le bootstrap démarre la session et vérifie le CSRF, puis le routeur appelle la méthode du contrôleur qui correspond au paramètre page. »

4. La base de données

MCD → MLD

Association	Cardinalités	Clé étrangère obtenue (côté 1,1)
utilisateurs **publier** annonces	0,n — 1,1	annonces.#id_proprietaire
utilisateurs **envoyer** candidatures	0,n — 1,1	candidatures.#id_gardien
annonces **viser** candidatures	0,n — 1,1	candidatures.#id_annonce
utilisateurs **rédiger** avis	0,n — 1,1	avis.#id_auteur
utilisateurs **recevoir** avis	0,n — 1,1	avis.#id_cible
annonces **concerner** avis	0,n — 1,1	avis.#id_annonce

- **Règle de passage** : pour une association 1,1 — 0,n, la clé primaire du côté 0,n devient clé étrangère dans la table du côté 1,1.
- Les noms id_auteur, id_cible… viennent des **rôles** donnés aux liens dans Looping.
- Deux associations entre utilisateurs et avis (rédiger / recevoir) : c'est ce qui distingue l'auteur de la personne notée.
- **ON DELETE CASCADE** : supprimer un compte supprime ses annonces, candidatures et avis.
- Statuts : annonces `ouverte → pourvue → terminee` ; candidatures `en_attente → acceptee | refusee`.
- `email UNIQUE` : MySQL refuse deux comptes avec le même e-mail (erreur 1062).
- Migrations (`sql/migrations/`) : modifient une base déjà remplie sans tout réimporter (001 ajoute ville, 002 renomme date en dates).

5. Les modèles — une phrase par méthode

Annonce (Idir)

Méthode	Ce qu'elle fait
rechercher()	Construit un SELECT avec des ? selon les filtres (mots-clés, animal, ville, propriétaire) ; sans propriétaire, seulement les annonces ouvertes.
creer()	Valide titre, type, dates (format, pas dans le passé, fin ≥ début), description, ville et propriétaire ; renvoie un tableau d'erreurs, vide = annonce insérée.
cloturer()	Vérifie que l'annonce existe, appartient à l'utilisateur et n'est pas déjà terminée, puis la passe en terminee ; le WHERE répète ces conditions pour qu'un double envoi ne fasse rien.
marquerPourvue()	Passe une annonce ouverte en pourvue ; renvoie true si une ligne a changé (prévue pour l'acceptation d'une candidature).
trouver() / toutes()	Une annonce par id (null si absente) / toutes les annonces, la plus récente d'abord.
villeDisponible()	Vérifie si la colonne ville existe, pour marcher même sur une base sans la migration.

Candidature (Ama)

Méthode	Ce qu'elle fait
pourUtilisateur()	Renvoie les candidatures envoyées par l'utilisateur et celles reçues sur ses annonces, avec les noms (jointures).
postuler()	Dans une transaction, verrouille l'annonce (FOR UPDATE), refuse si elle n'est pas accessible, si c'est sa propre annonce ou en cas de doublon, puis insère en en_attente.
decider()	Vérifie que l'utilisateur est le propriétaire et que la candidature est en_attente, puis la passe en acceptee ou refusee.
estAccessible() *(privée)*	Une annonce est accessible si elle est ouverte et que sa date de fin n'est pas passée.

Avis et Utilisateur (Maria)

Méthode	Ce qu'elle fait
Avis::pourAnnonce()	Liste les avis d'une annonce avec le nom de l'auteur et de la cible (deux jointures sur utilisateurs avec des alias).
Avis::destinataires()	Après la date de fin seulement : le propriétaire peut noter ses gardiens acceptés, un gardien accepté peut noter le propriétaire ; exclut les avis déjà déposés.
Avis::creer()	Dans une transaction, verrouille l'annonce, vérifie note 1–5, commentaire ≤ 2 000 caractères et cible autorisée, puis insère.
Utilisateur::creer()	Insère un compte avec le mot de passe haché par password_hash() (la validation est faite dans le contrôleur).
Utilisateur::trouverParEmail()	Renvoie id, nom et hash du mot de passe pour un e-mail, ou null.

6. Les contrôleurs — ce que fait chaque action

Action	Déroulement
AnnonceController::index	Lit les filtres de l'URL, appelle rechercher(), calcule les statistiques et la pagination (6 par page), affiche annonces.php. Mes annonces exige d'être connecté.
AnnonceController::creer	Visiteur → flash + connexion. POST : nettoie la saisie, appelle Annonce::creer() ; succès → flash + 303 ; erreurs → 422 et formulaire réaffiché avec la saisie.
AnnonceController::cloturer	POST uniquement (sinon 405) ; appelle Annonce::cloturer() avec l'id de la session ; flash de succès ou d'erreur puis redirection.
CandidatureController::postuler	Visiteur → mémorise l'annonce en session et l'envoie se connecter (il revient ensuite sur l'annonce). Vérifie annonce existante / pas la sienne / ouverte, puis postuler() ; doublon → 409.
CandidatureController::decision	POST uniquement ; appelle decider() ; refus → 403 et liste réaffichée avec l'erreur ; succès → flash « La décision a été enregistrée. »
AuthController::inscription	Valide nom, e-mail, mot de passe (8 à 72 octets) et confirmation ; e-mail déjà pris = erreur MySQL 1062 transformée en message.
AuthController::connexion	trouverParEmail() + password_verify() ; succès → session_regenerate_id(), nouvel id en session, nouveau jeton CSRF. Message d'erreur identique pour e-mail ou mot de passe faux.
AuthController::deconnexion	POST uniquement ; vide la session et régénère son identifiant.
AvisController::index	Annonce introuvable → 404 ; POST : valide note, commentaire, cible, puis Avis::creer() ; refus → 403.

7. Le socle commun : gabarit, e(), flash, PRG

- **Gabarit** : chaque vue définit $titre, charge header.php (CSS, favicon, navigation, messages flash), son contenu, puis footer.php. Toutes les pages ont donc le même style.
- **e()** = `htmlspecialchars($texte, ENT_QUOTES, 'UTF-8')` : < > & " ' deviennent du texte. Les nombres sont convertis avec (int).
- **Flash** : `ajouterFlash()` range un message en session avant une redirection ; le header l'affiche avec `lireFlash()`, qui le lit puis l'efface → il ne s'affiche qu'une fois.
- **Post/Redirect/Get** : après un POST réussi, redirection 303 vers une page GET ; rafraîchir ne renvoie pas le formulaire.
- **Saisie conservée** : en cas d'erreur, le contrôleur ne redirige pas : il réaffiche la vue avec les valeurs saisies (jamais les mots de passe).

8. La sécurité

Risque	Protection	Où
XSS (script injecté)	e() sur tout texte affiché	vues
Injection SQL	Requêtes préparées : prepare() avec des ?, puis execute([...])	modèles
CSRF (formulaire envoyé depuis un autre site)	Jeton random_bytes en session, caché dans chaque formulaire, comparé avec hash_equals() ; sinon 403	bootstrap
Mots de passe volés	password_hash() / password_verify() ; 72 octets max car bcrypt ignore la suite	Utilisateur, AuthController
Vol / fixation de session	session_regenerate_id() à la connexion et déconnexion ; cookie httponly, samesite	bootstrap, AuthController
Usurpation	L'auteur, le propriétaire, le gardien sont pris dans la session, jamais dans le formulaire	contrôleurs
Action d'un tiers	Les modèles vérifient le propriétaire (cloturer, decider) ; le WHERE le répète	modèles
Deux envois simultanés	Transaction + SELECT … FOR UPDATE : la ligne est verrouillée, pas de doublon	postuler(), Avis::creer()
Deviner un compte	Même message pour e-mail inconnu et mot de passe faux	connexion

9. Les codes HTTP utilisés

Code	Sens	Quand
303	Redirection vers une page GET	Après chaque action réussie (avec flash)
403	Interdit	Jeton CSRF faux, postuler à sa propre annonce, décision ou avis non autorisé
404	Introuvable	?page= inconnu, annonce inexistante
405	Méthode non autorisée	Ouvrir en GET une action réservée au POST (clôture, décision, déconnexion)
409	Conflit	Candidature en double
422	Données invalides	Formulaire d'annonce avec erreurs
503	Service indisponible	MySQL éteint ou config.php absent

10. Le parcours complet demandé par le prof

Étape	Ce qui se passe dans le code	État
1. Publier	creer-annonce → Annonce::creer() → flash	fait
2. Trouver et postuler	annonces + filtres → rechercher() ; postuler → Candidature::postuler()	fait
3. Valider le gardien	candidatures-recues → decision-candidature → decider()	fait
4. Pourvue + autres refusées	decider() doit appeler marquerPourvue() et refuser les autres candidatures	à faire (Ama)
5. Terminée	Bouton « Terminer la garde » → cloturer() ; date_fin passée en base pour autoriser les avis	fait, à tester
6. Avis + note moyenne sur le profil	Avis::creer() existe ; noteMoyenne() et la page profil sont à créer	à faire (Maria, Idir)

Ce qui va encore changer : Avis::noteMoyenne() (moyenne AVG, null sans avis), la page profil public, le refus automatique dans decider(), les messages flash d'inscription/connexion/avis (aujourd'hui inscrit=1 et ajoute=1) et la validation dans Utilisateur::creer(). Relire cette fiche après leur ajout.

11. Git et travail d'équipe

- Circuit : **feature/prénom-fonctionnalité** → **develop** → **main** ; seul Idir fusionne dans develop et main.
- Un hook pre-push refuse les mauvais noms de branche ; un workflow GitHub vérifie les pull requests.
- CHANGELOG daté (heure de Paris, prénom), ROADMAP des tâches, ORAL.md pour réviser, PROMPTS.md pour l'historique.
- config/config.php n'est jamais versionné (mot de passe MySQL).

12. Questions d'oral — essaie de répondre avant de lire la réponse

1. Que se passe-t-il avec ?page=nimportequoi ?
Le routeur ne trouve pas la clé dans $routes : code 404 et page « introuvable » avec le gabarit.

2. Pourquoi un seul index.php ?
Une seule porte d'entrée : session, CSRF et chargements sont faits une fois pour toutes les pages.

3. Pourquoi valider dans le modèle ?
C'est lui qui écrit en base : quel que soit l'appelant, une donnée invalide n'est jamais enregistrée.

4. Pourquoi une redirection 303 après un POST ?
Post/Redirect/Get : rafraîchir la page ne renvoie pas le formulaire une seconde fois.

5. Comment marche un message flash ?
Rangé en session avant la redirection, lu puis effacé par lireFlash() dans le header : affiché une seule fois.

6. Que fait htmlspecialchars ?
Transforme les caractères spéciaux HTML en texte : un <script> s'affiche au lieu de s'exécuter (XSS).

7. À quoi sert une requête préparée ?
La requête et les valeurs sont envoyées séparément : une saisie ne peut pas modifier le SQL.

8. Comment fonctionne le jeton CSRF ?
Jeton aléatoire en session, recopié dans chaque formulaire ; à chaque POST, le bootstrap compare les deux avec hash_equals().

9. Pourquoi le propriétaire vient-il de la session ?
Un champ caché du formulaire peut être modifié ; la session, non.

10. Pourquoi password_hash et pas MD5 ?
Hachage lent avec sel automatique ; password_verify compare sans jamais déchiffrer.

11. À quoi sert session_regenerate_id() ?
Change l'identifiant de session à la connexion : un identifiant volé avant ne sert plus (fixation de session).

12. Pourquoi une transaction et FOR UPDATE dans postuler() ?
Le verrou empêche deux envois simultanés de passer tous les deux le test du doublon.

13. Comment savoir si cloturer() a vraiment modifié l'annonce ?
rowCount() === 1 : une ligne a changé ; sinon elle était déjà terminée.

14. Qui peut laisser un avis, et quand ?
Après la date de fin : le propriétaire note ses gardiens acceptés, un gardien accepté note le propriétaire, une fois par personne.

15. Pourquoi candidature est une entité et pas une association ?
Elle a son identifiant et ses attributs (message, statut, date) et est reliée à deux entités.

16. D'où viennent id_auteur et id_cible ?
Des rôles donnés aux associations rédiger et recevoir dans le MCD ; sans rôle, Looping écrit id_1, id_2.

17. Que fait ON DELETE CASCADE ?
Supprimer une ligne parente supprime automatiquement les lignes qui la référencent.

18. Pourquoi le même message pour un mauvais e-mail ou mot de passe ?
Pour ne pas révéler quels e-mails ont un compte.

19. Comment la saisie est-elle conservée après une erreur ?
Le contrôleur réaffiche la vue sans rediriger ; la vue remet les valeurs avec value="<?= e(...) ?>".

20. Que voit-on si MySQL est éteint ?
Le routeur attrape la PDOException : code 503 et page « indisponible » qui explique quoi vérifier.